

2026 IBM CONSULTING ALL-DEVELOPER ICA BOB-A-THON

# Universal Spec MCP End-to-End Exercise

Making Bob Think Before It Codes

**Author:** Ahmed Elpanannan | **Version:** 3.0 | **Date:** May 2026

## Setup & Configuration

---

This section walks you through downloading, installing, and configuring the project from scratch. No prior experience with Python or MCP is required — just follow each step in order.

### 1 Download the Project

Go to the GitHub repository in your web browser:

```
https://github.ibm.com/Ahmed-Elpannann/Bob-A-Thon-2026-Spec-Driven
```

Click the green **"Code"** button, then click **"Download ZIP"**. Save the file to your computer (e.g., your Desktop or Downloads folder).

Once downloaded, right-click the ZIP file and select **"Extract All..."** (Windows) or double-click it (Mac). This creates a folder called `Bob-A-Thon-2026-Spec-Driven-main`.

### 2 Install Python (if not already installed)

Open a terminal (Command Prompt on Windows, Terminal on Mac) and type:

```
python --version
```

If you see `Python 3.11` or higher, you are good. If not, download Python from:

```
https://www.python.org/downloads/
```

**Important:** During installation on Windows, check the box that says **"Add Python to PATH"**.

### 3 Install the Required Packages

Open a terminal and navigate to the project folder you extracted:

```
cd Desktop/Bob-A-Thon-2026-Spec-Driven-main
```

Then install the dependencies:

```
pip install -e .
```

This installs the MCP server and all its dependencies (fastmcp, pydantic, etc.). You only need to do this once.

#### 4 Verify the Installation

Run the test suite to confirm everything works:

```
python -m pytest tests/ -v
```

You should see **81 tests passed** in green. If any test fails, check that Python 3.11+ is installed and try `pip install -e .` again.

#### 5 Open the Project in VS Code

Open VS Code, then click **File → Open Folder** and select the `Bob-A-Thon-2026-Spec-Driven-main` folder.

Bob (the IBM AI assistant) will automatically detect the `.bob/mcp.json` file in the project and connect to the MCP server.

#### 6 Start the MCP Server

The MCP server starts automatically when Bob detects the `.bob/mcp.json` configuration. If you need to start it manually for testing:

```
uvx fastmcp run src/universal_spec_mcp/server.py
```

You should see: `Universal Spec Architect running on stdio`

#### 7 Select the BMad Mode in Bob

In the Bob chat panel (bottom of VS Code), click the mode selector and choose **"BMad Spec Architect"**. Bob will confirm:

**BOB**

BMad Spec Architect mode activated. I have loaded four specialist skills: Analyst (Mary), Architect (Winston), PM (John), and Developer (Amelia). Tell me what feature you want to build.

#### Troubleshooting

- **Bob doesn't show the mode:** Make sure the `.bob/` folder is in the root of the project you opened.
- **"Module not found" error:** Run `pip install -e .` again from the project folder.
- **MCP server won't start:** Check that `uvx` is installed: `pip install uv`
- **Tests fail:** Ensure Python 3.11+ with `python --version`

# Overview

## Making Bob Think Before It Codes

AI coding assistants are powerful — but by default, they are reactive. When asked to build a feature, they immediately start writing code. In an enterprise environment, this leads to undocumented decisions, missed edge cases, and code that is difficult to maintain or audit. The Universal Spec MCP server solves this by inserting a structured engineering workflow before any code is written. It forces the AI to document requirements, produce a design, and break the work into tasks — ensuring that every line of code is traceable back to an approved specification.

## Workflow Architecture

The system operates across four distinct layers:

- **You (The Developer):** You provide the initial idea and answer clarifying questions.
- **BMad Agents:** Four specialized personas (Mary, Winston, John, Amelia) guide the conversation and execute specific phases of the workflow.
- **MCP Server:** A Python backend that exposes 25 tools to the agents, enforcing rules like EARS notation validation and privacy filtering.
- **Project Repository:** The `.specs/` directory where all requirements, designs, and tasks are saved as versioned Markdown files.

## The Exercise: Password Reset Flow

In this exercise, we will build a "User Password Reset Flow" from scratch. We will walk through all four phases, interacting with each agent in turn.

| Phase              | Agent   | Role                 | MCP Tool Used                   |
|--------------------|---------|----------------------|---------------------------------|
| 1. Analysis        | Mary    | Clarify requirements | <code>write_requirements</code> |
| 2. Architecture    | Winston | System design & ADRs | <code>write_design</code>       |
| 3. Story Breakdown | John    | Task definition      | <code>write_tasks</code>        |
| 4. Implementation  | Amelia  | Write code           | <code>update_task_status</code> |

## How to Read This Document

The following pages use color-coded boxes to show exactly what happens during the exercise:

### BLUE BOX

This is what you type into the Bob chat panel.

### TEAL BOX

This is how the BMad agent responds to you.

### PURPLE BOX

This shows the exact JSON payload the agent sends to the MCP server.

### ORANGE BOX

This is a note or explanation about what just happened.

## PHASE 1

# Mary (Analyst)

Requirements Gathering & EARS Notation

## STEP 1.1

### Activate Mary

#### YOU

Activate the bmad-analyst skill. I want to build a user password reset flow.

#### MARY (ANALYST)

Hello! I'm Mary. I'll help you define the requirements for the password reset flow. Before I write the specification, I need to clarify a few details:

1. How will the user receive the reset link? (e.g., Email, SMS)
2. How long should the reset token be valid?
3. Are there any specific security constraints? (e.g., password complexity)
4. What happens after a successful reset?

## STEP 1.2

**Provide Context****YOU**

1. Email via SendGrid.
2. Valid for 15 minutes.
3. Must be at least 12 characters with a number and symbol.
4. Redirect to the login page with a success message.

**MARY (ANALYST)**

Thank you. Here is the Product Brief:

**Feature:** Password Reset Flow

**Goal:** Allow users to securely reset their passwords via email.

**Constraints:** 15-minute token expiry, strict password complexity, SendGrid integration.

Does this look correct? If so, I will generate the formal EARS requirements.

**STEP 1.3****Approve and Generate Requirements****YOU**

Yes, that's correct. Generate the requirements.

## MCP TOOL CALL – WRITE\_REQUIREMENTS

```

{
  "feature_name": "password-reset-flow",
  "requirements": [
    {
      "id": "REQ-001",
      "title": "Reset Request Initiation",
      "ears_statement": "When a registered user submits their email address on the password reset page, the system shall generate a cryptographically secure token and send a reset link to that email via SendGrid.",
      "description": "Entry point for the reset flow. The system must validate the email exists before sending.",
      "acceptance_criteria": ["Email is validated against user database", "Token is 64-character hex string", "SendGrid API returns 202 Accepted", "Link contains token as URL parameter"]
    },
    {
      "id": "REQ-002",
      "title": "Token Expiry Enforcement",
      "ears_statement": "While a reset token exists, if more than 15 minutes have elapsed since its creation, the system shall reject the token and display an expiry message.",
      "description": "Time-based security constraint to limit the window of vulnerability.",
      "acceptance_criteria": ["Token created_at timestamp is stored", "Validation checks current_time - created_at > 15 min", "Expired tokens return HTTP 410 Gone", "User sees clear expiry message with option to request new link"]
    },
    {
      "id": "REQ-003",
      "title": "Password Complexity Validation",
      "ears_statement": "When the user submits a new password, the system shall reject it unless it contains at least 12 characters, including at least one numeric digit and one symbol.",
      "description": "Enforces organizational password policy at the point of reset.",
      "acceptance_criteria": ["Minimum 12 characters enforced", "At least one digit [0-9] required", "At least one symbol from !@#$$%^&*() required", "Clear error message on validation failure"]
    },
    {
      "id": "REQ-004",
      "title": "Successful Reset Redirect",
      "ears_statement": "When the password has been successfully updated, the system shall invalidate the token, redirect the user to the login page, and display a confirmation message.",
      "description": "Completes the flow and ensures the token cannot be reused.",
      "acceptance_criteria": ["Token is deleted from database after use", "HTTP 302 redirect to /login", "Flash message: 'Password updated successfully'", "Old password no longer works"]
    }
  ]
}

```



**WHAT JUST HAPPENED**

Mary called the `write_requirements` MCP tool. The server validated that each requirement uses proper EARS notation (When/While/If keywords), then saved the file to `.specs/password-reset-flow/requirements.md`.

**PHASE 2**

## Winston (Architect)

System Design & Architecture Decision Records

**STEP 2.1**

### Activate Winston

**YOU**

Switch to the `bmad-architect` skill. Design the password reset flow.

**WINSTON (ARCHITECT)**

I'm Winston. Let me first read the approved requirements to understand what I'm designing for.

**MCP TOOL CALL — READ\_SPEC**

```
{
  "feature_name": "password-reset-flow",
  "document_type": "requirements"
}
```

**WHAT JUST HAPPENED**

Winston called `read_spec` to load the requirements Mary wrote. This ensures his design is grounded in the approved specification — not guesswork.

## STEP 2.2

## Design Produced

### WINSTON (ARCHITECT)

Based on the requirements, here is my proposed design:

**Data Model:** New `password_reset_tokens` table with columns: `id`, `user_id`, `token_hash`, `created_at`, `used_at`.

**Sequence:** POST /reset-request → validate email → generate token → hash with SHA-256 → store → send via SendGrid → user clicks link → GET /reset/:token → validate expiry → show form → POST /reset/:token → validate password → update user → delete token → redirect.

#### ADR-001: Token Storage Strategy

Decision: Store SHA-256 hash of token, not plaintext.

Rationale: If the database is compromised, attackers cannot use stolen tokens. The token is only sent in the email — the server never stores the raw value.

Alternatives considered: (1) Store plaintext — rejected due to security risk. (2) Encrypt with AES — rejected as unnecessary complexity when hashing is sufficient.

## MCP TOOL CALL – WRITE\_DESIGN

```
{
  "feature_name": "password-reset-flow",
  "design_sections": [
    {
      "title": "Data Model",
      "content": "New table `password_reset_tokens`: \n- id: UUID primary key\n- user_id: FK to users.id\n- token_hash: VARCHAR(64) – SHA-256 of the raw token\n- created_at: TIMESTAMP\n- used_at: TIMESTAMP nullable"
    },
    {
      "title": "API Endpoints",
      "content": "POST /api/reset-request – accepts {email}, returns 200 regardless (prevents enumeration)\nGET /reset/:token – validates token, renders password form\nPOST /reset/:token – accepts {password, confirm}, updates user, deletes token"
    },
    {
      "title": "Sequence Diagram",
      "content": "User → POST /reset-request → Server validates email → Generate 32-byte random token → SHA-256 hash → Store hash in DB → Send raw token via SendGrid → User clicks link → GET /reset/:token → Server hashes URL token → Compare with DB → Check created_at < 15min → Render form → User submits → POST /reset/:token → Validate password complexity → Update users.password_hash → Delete token row → Redirect /login"
    },
    {
      "title": "ADR-001: Token Storage Strategy",
      "content": "Decision: Store SHA-256 hash of token, not plaintext.\nStatus: Accepted.\nRationale: Database compromise does not expose usable tokens.\nAlternatives: (1) Plaintext – rejected (security). (2) AES encryption – rejected (unnecessary complexity)."

```

## PHASE 3

## John (Product Manager)

Story Breakdown &amp; Task Definition

**STEP 3.1****Activate John****YOU**

Switch to the bmad-pm skill. Break the password reset flow into tasks.

**JOHN (PM)**

I'm John. Let me read both the requirements and the design to create a proper task breakdown.

**MCP TOOL CALLS — READ\_SPEC (×2)**

```
{  
  "feature_name": "password-reset-flow",  
  "document_type": "requirements"  
}  
{  
  "feature_name": "password-reset-flow",  
  "document_type": "design"  
}
```

**STEP 3.2****Task Breakdown****JOHN (PM)**

Here are four tasks, ordered by dependency. Each task is independently testable and maps to specific requirements:

**TASK-001:** Database migration (no dependencies)

**TASK-002:** Reset request endpoint (depends on TASK-001)

**TASK-003:** Token validation & password update endpoint (depends on TASK-001)

**TASK-004:** Integration test suite (depends on TASK-002, TASK-003)

## MCP TOOL CALL – WRITE\_TASKS

```

{
  "feature_name": "password-reset-flow",
  "tasks": [
    {
      "id": "TASK-001",
      "title": "Create password_reset_tokens migration",
      "description": "Write the database migration to create the password_reset_tokens table with all columns from the design.",
      "depends_on": [],
      "acceptance_criteria": ["Migration creates table with id, user_id, token_hash, created_at, used_at", "Migration is reversible", "Index on token_hash for fast lookup"]
    },
    {
      "id": "TASK-002",
      "title": "Implement POST /api/reset-request",
      "description": "Build the endpoint that accepts an email, generates a token, stores the hash, and sends the email via SendGrid.",
      "depends_on": ["TASK-001"],
      "acceptance_criteria": ["Returns 200 for both valid and invalid emails", "Generates 32-byte cryptographically random token", "Stores SHA-256 hash in DB", "Sends email via SendGrid with reset link"]
    },
    {
      "id": "TASK-003",
      "title": "Implement GET/POST /reset/:token",
      "description": "Build the token validation endpoint and the password update endpoint with complexity validation.",
      "depends_on": ["TASK-001"],
      "acceptance_criteria": ["GET validates token hash and checks 15-min expiry", "Returns 410 for expired tokens", "POST validates password complexity (12+ chars, digit, symbol)", "Updates user password hash and deletes token", "Redirects to /login with success message"]
    },
    {
      "id": "TASK-004",
      "title": "End-to-end integration tests",
      "description": "Write integration tests covering the full happy path and all error cases.",
      "depends_on": ["TASK-002", "TASK-003"],
      "acceptance_criteria": ["Happy path: request → email → click → reset → login works", "Expired token returns 410", "Weak password returns 400 with clear message", "Reused token returns 404", "Non-existent email still returns 200"]
    }
  ]
}

```

## PHASE 4

## Amelia (Developer)

Implementation &amp; Task Tracking

## STEP 4.0

## Orientation — Check Project State

## YOU

Switch to the bmad-dev skill. Implement the password reset flow.

## AMELIA (DEVELOPER)

I'm Amelia. Let me first check what specs exist for this project.

## MCP TOOL CALL — LIST\_SPECS

```
{}
```

## AMELIA (DEVELOPER)

Found feature: `password-reset-flow` with requirements, design, and tasks all present. Reading all three now.

## MCP TOOL CALLS — READ\_SPEC (×3)

```
{"feature_name": "password-reset-flow", "document_type": "requirements"}  
{"feature_name": "password-reset-flow", "document_type": "design"}  
{"feature_name": "password-reset-flow", "document_type": "tasks"}
```

## STEP 4.1

## Implement TASK-001 — Database Migration

## MCP TOOL CALL — RUN\_HOOK (PRE\_TASK)

```
{"feature_name": "password-reset-flow", "hook_type": "pre_task", "task_id": "TASK-001"}
```

## WHAT JUST HAPPENED

The pre\_task hook checks that all dependencies for TASK-001 are completed. Since TASK-001 has no dependencies, it passes immediately.

## MCP TOOL CALL — UPDATE\_TASK\_STATUS

```
{"feature_name": "password-reset-flow", "task_id": "TASK-001", "status": "in_progress"}
```

## AMELIA (DEVELOPER)

Writing the migration file now:

```
-- migrations/001_create_password_reset_tokens.sql
CREATE TABLE password_reset_tokens (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  token_hash VARCHAR(64) NOT NULL,
  created_at TIMESTAMP NOT NULL DEFAULT NOW(),
  used_at TIMESTAMP
);

CREATE INDEX idx_token_hash ON password_reset_tokens(token_hash);
```

## MCP TOOL CALL — UPDATE\_TASK\_STATUS

```
{"feature_name": "password-reset-flow", "task_id": "TASK-001", "status": "completed"}
```

## MCP TOOL CALL — RUN\_HOOK (POST\_TASK)

```
{"feature_name": "password-reset-flow", "hook_type": "post_task", "task_id": "TASK-001",
"summary": "Created migration for password_reset_tokens table with SHA-256 token_hash
column and index."}
```



## STEP 4.2

## Implement TASK-002 — Reset Request Endpoint

## MCP TOOL CALL — RUN\_HOOK (PRE\_TASK)

```
{"feature_name": "password-reset-flow", "hook_type": "pre_task", "task_id": "TASK-002"}
```

## WHAT JUST HAPPENED

The pre\_task hook checks TASK-002's dependency (TASK-001). Since TASK-001 is completed, it passes.

## MCP TOOL CALL — UPDATE\_TASK\_STATUS

```
{"feature_name": "password-reset-flow", "task_id": "TASK-002", "status": "in_progress"}
```

## AMELIA (DEVELOPER)

Writing the reset request endpoint:

```
import secrets, hashlib
from sendgrid import SendGridAPIClient
from sendgrid.helpers.mail import Mail

@app.post("/api/reset-request")
async def reset_request(email: str):
    user = await db.get_user_by_email(email)
    if not user:
        return {"message": "If that email exists, a reset link was sent."}

    token = secrets.token_hex(32) # 64-char hex string
    token_hash = hashlib.sha256(token.encode()).hexdigest()

    await db.insert_reset_token(user.id, token_hash)

    link = f"https://app.example.com/reset/{token}"
    msg = Mail(to_emails=email, subject="Password Reset",
              html_content=f'<a href="{link}">Reset your password</a>')
    SendGridAPIClient(SENDGRID_KEY).send(msg)

    return {"message": "If that email exists, a reset link was sent."}
```

## MCP TOOL CALL — UPDATE\_TASK\_STATUS

```
{"feature_name": "password-reset-flow", "task_id": "TASK-002", "status": "completed"}
```

## STEP 4.3

## Implement TASK-003 — Token Validation &amp; Password Update

## MCP TOOL CALLS

```
run_hook: {"feature_name": "password-reset-flow", "hook_type": "pre_task", "task_id": "TASK-003"}
update_task_status: {"feature_name": "password-reset-flow", "task_id": "TASK-003", "status": "in_progress"}
```

## AMELIA (DEVELOPER)

Writing the token validation and password update endpoints:

```
import re
from datetime import datetime, timedelta

@app.get("/reset/{token}")
async def validate_token(token: str):
    token_hash = hashlib.sha256(token.encode()).hexdigest()
    record = await db.get_reset_token(token_hash)

    if not record:
        raise HTTPException(404, "Invalid reset link.")
    if datetime.utcnow() - record.created_at > timedelta(minutes=15):
        await db.delete_reset_token(token_hash)
        raise HTTPException(410, "This reset link has expired. Please request a new one.")

    return render_template("reset_form.html", token=token)

@app.post("/reset/{token}")
async def reset_password(token: str, password: str, confirm: str):
    if password != confirm:
        raise HTTPException(400, "Passwords do not match.")
    if len(password) < 12 or not re.search(r'\d', password) or not re.search(r'[@#$$%^&*()]\'', password):
        raise HTTPException(400, "Password must be 12+ chars with a number and symbol.")

    token_hash = hashlib.sha256(token.encode()).hexdigest()
    record = await db.get_reset_token(token_hash)
    if not record or datetime.utcnow() - record.created_at > timedelta(minutes=15):
        raise HTTPException(410, "Token expired.")

    await db.update_user_password(record.user_id, hash_password(password))
    await db.delete_reset_token(token_hash)

    return RedirectResponse("/login?msg=Password+updated+successfully", status_code=302)
```

## MCP TOOL CALL — UPDATE\_TASK\_STATUS

```
{"feature_name": "password-reset-flow", "task_id": "TASK-003", "status": "completed"}
```

STEP 4.4

Implement TASK-004 — Integration Tests

MCP TOOL CALLS

```
run_hook: {"feature_name": "password-reset-flow", "hook_type": "pre_task", "task_id": "TASK-004"}
update_task_status: {"feature_name": "password-reset-flow", "task_id": "TASK-004", "status": "in_progress"}
```

WHAT JUST HAPPENED

The pre\_task hook checks TASK-004's dependencies (TASK-002 and TASK-003). Both are completed, so it passes.

AMELIA (DEVELOPER)

Writing the integration test suite... [tests written]

MCP TOOL CALLS — COMPLETION

```
update_task_status: {"feature_name": "password-reset-flow", "task_id": "TASK-004", "status": "completed"}
run_hook: {"feature_name": "password-reset-flow", "hook_type": "post_task", "task_id": "TASK-004", "summary": "All 5 integration tests passing: happy path, expired token, weak password, reused token, non-existent email."}
```

STEP 4.5

Privacy Filter in Action

BEHIND THE SCENES

Throughout implementation, Amelia's code contained the SendGrid API key SG.abc123... . The MCP server's privacy filter automatically detected and redacted it before writing to .specs/. The stored file shows [REDACTED\_SENDGRID\_KEY] instead of the real value. This happens transparently — no action required from you.

Summary & Reference

What Was Produced

| File                                       | Created By | Contains                        |
|--------------------------------------------|------------|---------------------------------|
| .specs/password-reset-flow/requirements.md | Mary       | 4 EARS-validated requirements   |
| .specs/password-reset-flow/design.md       | Winston    | Data model, API design, ADR-001 |

|                                                   |        |                                 |
|---------------------------------------------------|--------|---------------------------------|
| <code>.specs/password-reset-flow/tasks.md</code>  | John   | 4 tasks with dependencies       |
| <code>.specs/password-reset-flow/meta.json</code> | System | Timestamps, versions, approvals |

### MCP Tool Reference

| Tool                                 | Used By | Purpose                                                |
|--------------------------------------|---------|--------------------------------------------------------|
| <code>initialize_spec</code>         | Any     | Create a new feature spec directory                    |
| <code>write_requirements</code>      | Mary    | Write EARS-validated requirements                      |
| <code>write_design</code>            | Winston | Write design document with ADRs                        |
| <code>write_tasks</code>             | John    | Write task breakdown with dependencies                 |
| <code>read_spec</code>               | All     | Read any spec document for a feature                   |
| <code>list_specs</code>              | All     | List all features in the project                       |
| <code>update_task_status</code>      | Amelia  | Mark tasks as todo/in_progress/completed/blocked       |
| <code>run_hook</code>                | Amelia  | Pre/post task dependency checks and memory storage     |
| <code>add_directive</code>           | Any     | Add project-wide rules (e.g., "always use TypeScript") |
| <code>remove_directive</code>        | Any     | Remove a directive by ID                               |
| <code>list_directives</code>         | Any     | View all active project directives                     |
| <code>store_memory</code>            | Any     | Save a decision or learning for future reference       |
| <code>get_memories_by_feature</code> | Any     | Retrieve memories relevant to a feature                |
| <code>health_check</code>            | Any     | Verify spec integrity and EARS compliance              |
| <code>search_specs</code>            | Any     | Full-text search across all specifications             |

### Task Status Markers

| Marker           | Status      | Meaning                           |
|------------------|-------------|-----------------------------------|
| <code>[ ]</code> | todo        | Not yet started                   |
| <code>[~]</code> | in_progress | Currently being worked on         |
| <code>[x]</code> | completed   | Done and verified                 |
| <code>[!]</code> | blocked     | Cannot proceed — dependency issue |

## If You Get Lost

### Use bmad-help at any time

If you are unsure what to do next, type: **"bmad-help — what should I do next?"**

The bmad-help skill will call `list_specs()` to see what features exist, then `read_spec()` on each document type to check what has been written. It returns an orientation report like:

"Feature password-reset-flow: requirements ✓, design ✓, tasks ✓, implementation 2/4 complete. → Next: activate bmad-dev and continue with TASK-003."

Follow the recommendation exactly. Do not skip phases or documents that are missing — the MCP server will reject the attempt with a phase gate error.

## Key Principles

- **No code without a spec.** The MCP server enforces this structurally — you cannot skip phases.
- **Every decision is documented.** ADRs, EARS requirements, and task acceptance criteria create a full audit trail.
- **Privacy is automatic.** API keys and secrets are redacted before they reach disk.
- **Memory persists.** Decisions stored via `store_memory` are available in future sessions.